

Práctica 1

Implementación de algoritmos paralelos de datos en GPU usando CUDA

Parte 1. Algoritmo de Floyd

Primero declaramos la matriz de entrada del problema como memoria fijada, lo que implica cambiar la manera en la que reservamos la memoria en el programa

Se ha cambiado la instruccion

```
checkCuda(cudaMalloc((void **) &d_In_M, size));
```

Por esta

```
checkCuda(cudaMallocHost((void **) &d_In_M, size));
```

Fijando la memoria en el host

Ahora bien, la implementación del kernel que utiliza grid bidimensional y bloques bidimensionales queda tal que así.

Se han declarado dos índices de fila y columna y se ha obtenido el nuevo índice 1D a partir de estos.

```
__global__ void floyd_kernel_2D(int *M, const int nverts, const int k)
{
    int j = blockIdx.x * blockDim.x + threadIdx.x;
    int i = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < nverts && j < nverts && i != j && i != k && j != k) {
        int ij = i * nverts + j;
        int Mij = M[ij];
        int Mikj = M[i * nverts + k] + M[k * nverts + j];
        M[ij] = (Mij > Mikj) ? Mikj : Mij;
    }
}
```

Y se llamaría de la siguiente manera

```
const int nverts = G.vertices;
const int niters = nverts;
const int nverts2 = nverts * nverts;

int *c_Out_M = new int[nverts2];
int size = nverts2 * sizeof(int);
int *d_In_M = NULL;

dim3 threadsPerBlock(ceil(sqrt(BLOCKSIZE)), ceil(sqrt(BLOCKSIZE)));
dim3 blocksPerGrid(
    ceil((float) nverts / threadsPerBlock.x),
    ceil((float) nverts / threadsPerBlock.y)
);

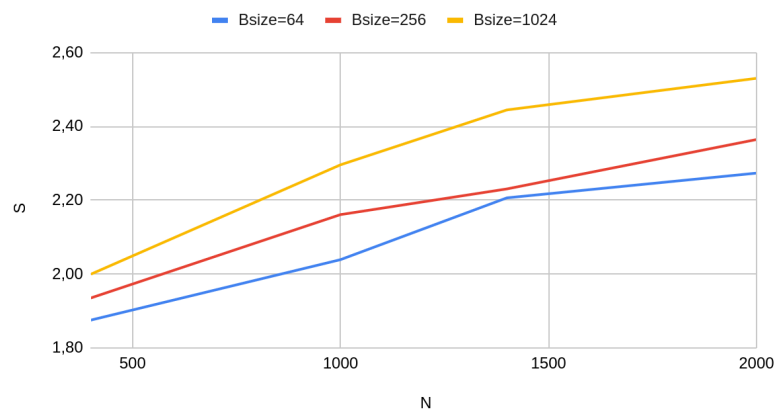
checkCuda(cudaMemcpy(d_In_M, A, size, cudaMemcpyHostToDevice));
for (int k = 0; k < niters; k++)
{
    floyd_kernel_2D<<<blocksPerGrid, threadsPerBlock>>>(d_In_M, nverts, k);
    checkCuda(cudaGetLastError());
}
checkCuda(cudaMemcpy(c_Out_M, d_In_M, size, cudaMemcpyDeviceToHost));

Tgpu = (clock() - time) / CLOCKS_PER_SEC;
cout << "Time spent on GPU= " << Tgpu << endl << endl;
```

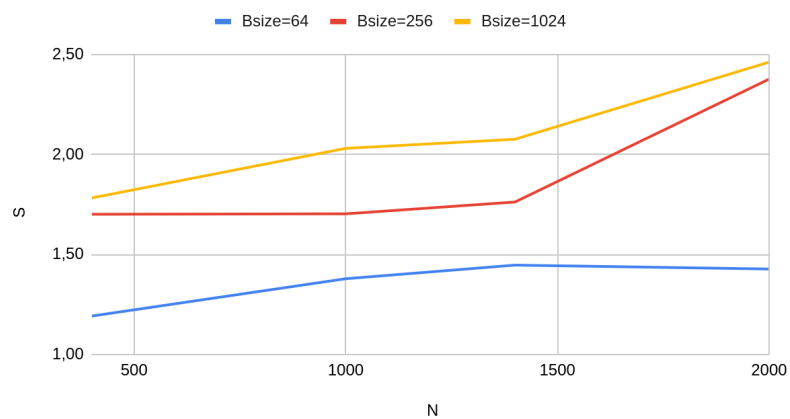
A continuacion se muestran los tiempos de ejecucion y ganancias de velocidad, comparando la version secuencial, la version del kernel 1D y 2D

BLOCKSIZE	N	TCPU	TGPU1D	SGPU1D	TGPU2D	SGPU2D
64	400	0,07355	0,039222	1,88	0,06170	1,19
64	1000	1,11722	0,54794	2,04	0,81039	1,38
64	1400	3,19139	1,44645	2,21	2,20602	1,45
64	2000	9,24228	4,06553	2,27	6,47619	1,43
256	400	0,07087	0,03662	1,94	0,04167	1,70
256	1000	1,10149	0,50968	2,16	0,64680	1,70
256	1400	3,03821	1,36200	2,23	1,72435	1,76
256	2000	9,24174	3,90862	2,36	3,88966	2,38
1024	400	0,07162	0,03582	2,00	0,04020	1,78
1024	1000	1,09830	0,47837	2,30	0,54117	2,03
1024	1400	3,07926	1,25962	2,44	1,48396	2,08
1024	2000	9,18841	3,63155	2,53	3,73459	2,46

Ganancia frente a N (version 1D)



Ganancia frente a N (version 2D)



El kernel de reducción de la matriz es el siguiente

Se ha implementado una triple reducción para obtener la suma, el mínimo y el máximo de cada bloque, esto es, usar por cada bloque, un trozo de memoria compartida equivalente a 3 bloques.

```
__global__ void reducir(int *M_in, const int nverts) {
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Suma
    sdata[tid] = (i < nverts*nverts && M_in[i] < INF) ? M_in[i] : 0;
    // Minimo
    sdata[blockDim.x+tid] = (i < nverts*nverts && M_in[i] > 0) ? M_in[i] : INF;
    // Maximo
    sdata[2*blockDim.x+tid] = (i < nverts*nverts && M_in[i] < INF) ? M_in[i] : 0;

    __syncthreads();

    // Reduccion de los datos del bloque
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            // Suma
            sdata[tid] += sdata[tid + s];
            // Minimo
            sdata[blockDim.x+tid] = (sdata[blockDim.x+tid] > sdata[blockDim.x+tid + s]) ?
                sdata[blockDim.x+tid + s] : sdata[blockDim.x+tid];
            // Maximo
            sdata[2*blockDim.x+tid] = (sdata[2*blockDim.x+tid] < sdata[2*blockDim.x+tid + s]) ?
                sdata[2*blockDim.x+tid + s] : sdata[2*blockDim.x+tid];
        }
        __syncthreads();
    }

    if (tid == 0) {
        // Se guarda suma, minimo y maximo de cada bloque de manera consecutiva
        // en la matriz de entrada (operacion terminal)
        M_in[3 * blockIdx.x] = sdata[0];
        M_in[3 * blockIdx.x + 1] = sdata[blockDim.x];
        M_in[3 * blockIdx.x + 2] = sdata[2*blockDim.x];
    }
}
```

Y se llamaría de la siguiente manera

```
dim3 threadsPerBlock_r(nverts, 1);
dim3 numBlocks_r(nverts, 1);
int smemSize = threadsPerBlock_r.x * 3 * sizeof(int);
reducir<<<numBlocks_r, threadsPerBlock_r, smemSize>>>(d_In_M, nverts);
checkCuda(cudaMemcpy(c_Out_M, d_In_M, size, cudaMemcpyDeviceToHost));
cudaDeviceSynchronize();

long int sum = 0;
float min = INF, max = 0;
for (int i=0; i < numBlocks_r.x; i++) {
    sum += c_Out_M[3*i];
    min = (c_Out_M[3*i+1] < min) ? c_Out_M[3*i+1] : min;
    max = (c_Out_M[3*i+2] > max) ? c_Out_M[3*i+2] : max;
}
float media = sum / numBlocks_r.x;
```

Captura de pantalla del algoritmo de Floyd implementando las 3 funcionalidades adicionales, con una entrada de 2000 vertices y Bsize=64

```
lu1smgb@kubuntu:~/Escritorio/PPR/P1/Floyd/v3$ ./floyd input/input2000
Using Device 0
.....
*****
Using Device 0: NVIDIA GeForce RTX 3050 Laptop GPU with CUDA Compute Capability 8.6
*****

***** Warming up GPU!!!
Tamaño de M: (2000, 2000)
Tamaño de bloque: (8, 8)
Tamaño de grid: (250, 250)
Time spent on GPU= 6.5262

Time spent on CPU= 9.49237

.....

Speedup TCPU/TGPU= 1.4545
.....

.....
WELL DONE!!! No errors found .....
.....

--- Estadísticas ---
suma = 3728838
media = 1864
minimo = 900
maximo = 2779
lu1smgb@kubuntu:~/Escritorio/PPR/P1/Floyd/v3$
```

Parte 2. Operacion vectorial

Se han implementado 4 kernels en total (3 por version de programa), uno por etapa de calculo (calculo de C, reduccion del máximo y obtencion de las sumas de cada bloque)

El kernel que calcula C usando `__shared__` funciona de manera muy similar al kernel de reduccion de la parte 1. La memoria compartida es igual de grande a 3 bloques, que guarda la región de las matrices A, B, C con las que trabajará cada bloque. El cálculo se realiza en la memoria compartida y finalmente, cuando se termine de calcular un valor C_i se guardará en la memoria global

```
// Kernel que calcula el vector C version __shared__
__global__ void vectorialGPU_shared(float *A, float *B, float *C, int N)
{
    extern __shared__ float cdata[]; // Bsize * sizeof(float) * 3 (A,B y C)

    int tid = threadIdx.x;
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    if (i < N)
    {
        cdata[tid] = A[i]; // A
        cdata[blockDim.x + tid] = B[i]; // B
        cdata[2 * blockDim.x + tid] = 0; // C
        __syncthreads();

        // Calculamos valor de C[i]
        for (int j = 0; j < blockDim.x; j++)
        {
            float c1 = cdata[j] * i;
            float c2 = cdata[blockDim.x + j];
            float v;
            if ( (int)ceil(c1) % 2 == 0 )
                v = c1 + c2;
            else
                v = c1 - c2;
            cdata[2*blockDim.x+tid] += v;
            __syncthreads();
        }
        C[i] = cdata[2*blockDim.x+tid];
    }
}
```

```
int numBlocks = ceil((float)N / M);
vectorialGPU_noshared<<<numBlocks, M>>>(A, B, C, N);
```

El kernel que no usa `__shared__` es más sencillo, pero tenemos que definir los límites con los que trabajarán cada uno de los bloques

```
// Kernel que calcula el vector C version sin utilizar __shared__
__global__ void vectorialGPU_noshared(float *A, float *B, float *C, int N)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N)
    {
        C[i] = 0;
        // Establecemos j y su valor limite
        int inicio = blockDim.x * blockIdx.x;
        int fin = inicio + blockDim.x;
        // Calculamos valor de C[i]
        for (int j = inicio; j < fin; j++)
        {
            float c1 = A[j] * i;
            float c2 = B[j];
            float v;
            if ( (int)ceil(c1) % 2 == 0 )
                v = c1 + c2;
            else
                v = c1 - c2;
            C[i] += v;
        }
    }
}
```

```
int numBlocks = ceil((float)N / M);
vectorialGPU_noshared<<<numBlocks, M>>>>(A, B, C, N);
```

Kernel de reducción a máximo

```
__global__ void reducirMaximos(float *C, float *D, int N) {
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    sdata[tid] = (i < N) ? C[i] : 0;
    __syncthreads();
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            if (sdata[tid + s] > sdata[tid]) {
                sdata[tid] = sdata[tid+s];
            }
        }
        __syncthreads();
    }
    if (tid == 0) {
        D[blockIdx.x] = sdata[0];
    }
}
```

```
unsigned int nB_byte = numBlocks * sizeof(float);
reducirMaximos<<<numBlocks, M, nB_byte>>>>(C, D, N);
```

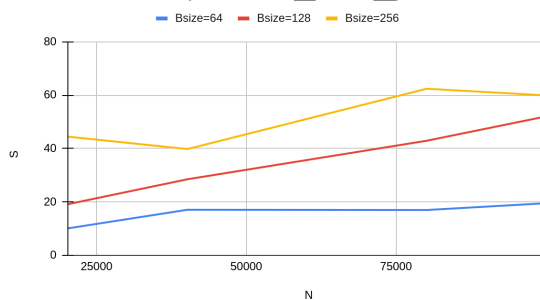

Kernel para calcular D

```
__global__ void reducirSumas(float *C, float *D, int N) {
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    sdata[tid] = (i < N) ? C[i] : 0;
    __syncthreads();
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }
    if (tid == 0) {
        D[blockIdx.x] = sdata[0];
    }
}
```

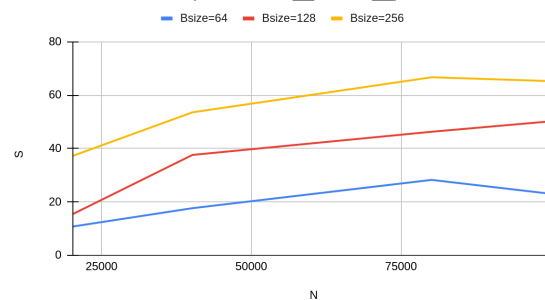
```
reducirSumas<<<numBlocks, M, nB_byte>>>(C, D, N);
```

BLOCKSIZE	N	TCPU	TGPU_shared	SGPU_shared	TGPU_nshared	SGPU_nshared
64	20224	0,002323	2,30E-04	10,10	0,000215	10,80
64	40192	0,004541	2,66E-04	17,07	0,000257	17,67
64	80128	0,009103	5,36E-04	16,98	0,000322	28,27
64	100096	0,011540	5,91E-04	19,53	0,000501	23,03
128	20224	0,004502	2,35E-04	19,16	0,000291	15,47
128	40192	0,008690	3,05E-04	28,49	0,000231	37,62
128	80128	0,017685	4,12E-04	42,92	0,000382	46,30
128	100096	0,022748	4,37E-04	52,05	0,000453	50,22
256	20224	0,008758	1,97E-04	44,46	0,000235	37,27
256	40192	0,017036	4,28E-04	39,80	0,000318	53,57
256	80128	0,033820	5,42E-04	62,40	0,000507	66,71
256	100096	0,042421	7,08E-04	59,92	0,000650	65,26

Ganancia frente a N (version con __shared__)



Ganancia frente a N (version sin __shared__)



Ejemplo de ejecución de la versión __shared__ con N=100096 y Bsize=256

```
lu1smgb@kubuntu:~/Escritorio/PPR/P1/Vectorial/Shared$ ./vectorial_shared 100096 256
*****
Using Device 0: NVIDIA GeForce RTX 3050 Laptop GPU with CUDA Compute Capability 8.6
*****

N (tamaño de los vectores) = 100096
M (tamaño de los bloques) = 256
Tiempo GPU (shared): 0.000739 seg
Tiempo CPU: 0.043553 seg
Speedup (shared): 58.935
Maximo GPU (shared): 1.40735e+07
Maximo CPU: 1.40735e+07
lu1smgb@kubuntu:~/Escritorio/PPR/P1/Vectorial/Shared$
```

Ejemplo de ejecución de la versión sin __shared__ con N=100096 y Bsize=256

```
lu1smgb@kubuntu:~/Escritorio/PPR/P1/Vectorial/NoShared$ ./vectorial_noshared 100096 256
*****
Using Device 0: NVIDIA GeForce RTX 3050 Laptop GPU with CUDA Compute Capability 8.6
*****

N (tamaño de los vectores) = 100096
M (tamaño de los bloques) = 256
Tiempo GPU (no shared): 0.000716 seg
Tiempo CPU: 0.042724 seg
Speedup (noshared): 59.6704
Maximo GPU (noshared): 1.40735e+07
Maximo CPU: 1.40735e+07
lu1smgb@kubuntu:~/Escritorio/PPR/P1/Vectorial/NoShared$
```

Conclusiones

Algoritmo de Floyd

Las ganancias obtenidas son mejores en la version 1D con respecto a la 2D y además tiene mejor evolución (excepto para Bsize=1024 por una tendencia de crecimiento muy alta), aunque en algunos casos la ganancia puede llegar a ser similar (p.e N = 2000 y Bsize = 256 ambos son aproximadamente 2.3)

Medias aritméticas

BLOCKSIZE	SGPU1D	SGPU2D
64	2,10	1,36
256	2,17	1,89
1024	2,32	2,09

Operacion vectorial

La evolución de las ganancias en ambas implementaciones es similar, pero obtiene, por lo general, valores mayores en la versión sin memoria compartida

Medias aritméticas

BLOCKSIZE	SGPU_shared	SGPU_nshared
64	15,92	19,94
128	35,66	37,40
256	51,64	55,70